# Laboratory 2 Report

ANDREA BOTTICELLA*, s347291, Politecnico di Torino, Italy
ELIA INNOCENTI*, s345388, Politecnico di Torino, Italy
SIMONE ROMANO*, s344024, Politecnico di Torino, Italy

This work addresses malware detection through behavioral analysis of API call sequences. We implement and compare four classification approaches: a frequency-based logistic regression baseline, Feed-Forward Neural Networks with learned embeddings, Recurrent Neural Networks (RNN, BiRNN, LSTM) for sequential modeling, and Graph Neural Networks (GCN, GraphSAGE, GAT) that represent call sequences as transition graphs. Experiments are conducted on a dataset of execution traces containing 258 unique API calls, with attention to class imbalance handling and variable-length sequence processing.

## 1 INTRODUCTION

Dynamic malware analysis leverages runtime behavior to detect malicious software, circumventing the limitations of static signature-based methods against obfuscated or polymorphic variants. API call sequences—the ordered invocations a program makes to the operating system—provide a rich behavioral fingerprint, capturing memory operations, registry accesses, and system queries that reveal malicious intent.

This laboratory implements a comparative study of neural network architectures for binary malware classification based on API call sequences. The dataset consists of execution traces, each represented as a variable-length sequence of API function names (e.g., `NtAllocateVirtualMemory`, `RegOpenKeyExW`).

We evaluate four progressively complex approaches:

- **Task 1: Frequency-based baseline** – implement and evaluate a logistic regression reference model.
- **Task 2: Feed-Forward Neural Networks** – compare fixed-size sequential IDs and learnable embeddings.
- **Task 3: Recurrent Neural Networks** – model ordered traces with recurrent architectures.
- **Task 4: Graph Neural Networks** – exploit transition graphs built from API call sequences.

The following sections detail the methodology and results for each task, analyzing how architectural choices impact classification performance.

## 2 TASK 1: FREQUENCY-BASED BASELINE

This section establishes a baseline for malware classification using a frequency-based representation of API call sequences. The intent is to quantify how far one can go with a simple model that ignores temporal order, and to provide a reference point for the more expressive architectures introduced later.

### 2.1 Data Loading and Exploration

The dataset is provided in JSON format, with each sample containing two fields: `api_call_sequence` (a list of API function names observed during execution) and `is_malware` (a binary label). Both training and test partitions are loaded for an initial inspection of size, label distribution, and vocabulary.

The training set contains **16,325 samples**, while the test set comprises **6,505 samples**. A striking observation emerges from examining the class distribution: malware samples constitute **96.26%** of both partitions (15,714 training and 6,262 test samples), leaving only 3.74% for benign software (611 training and 243 test samples). This severe class imbalance has important implications for model evaluation. Accuracy alone would be misleading, as a trivial classifier predicting "malware" for every sample would achieve 96% accuracy without learning anything meaningful. We therefore pay close attention to precision, recall, and F1-score for both classes, particularly monitoring performance on the minority benign class.

### 2.2 Vocabulary Extraction and Analysis

**Vocabulary and unseen calls.** The frequency-based representation requires a fixed vocabulary. To avoid any information leakage, the vocabulary is extracted from the **training set only** and used to define the feature space at inference time. The training set contains **258 unique API calls**, while the test set contains **232**.

This immediately exposes a vocabulary mismatch. Twenty-nine API calls appear only in the training set (they can influence training but never occur at test time), while **three appear exclusively in the test set**: `NtDeleteKey`,

*The authors collaborated closely in developing this project.

Publication date: June 2026.

`ControlService`, and `WSASocketA`. The latter case is the critical one, because the classifier would be asked to process signals that were not available during training.

**Handling unknown API calls.** The **test vocabulary is not used** to build the test dataframe, because doing so would leak information from the held-out partition. Unseen API calls can be handled by introducing an explicit unknown feature (e.g., `<UNK>`) or by discarding them when building the input vector. Since the three unseen calls appear in only three test samples, they have negligible impact at the dataset scale; the pipeline therefore ignores them and builds test vectors strictly over the training vocabulary.

### 2.3 Frequency-Based Feature Vectors

**Sparsity and ordering.** Each API call sequence is transformed into a **258-dimensional vector**, where each element is the count of occurrences of a specific API call from the training vocabulary. This bag-of-words representation **discards temporal ordering**: any permutation of the same multiset of calls maps to the same vector, and patterns that depend on call order are not represented.

The resulting feature matrices are **highly sparse**, as individual execution traces invoke only a small subset of the full API vocabulary. On average, training samples contain **21.95 non-zero elements per row**, representing just **8.51%** of the 258 dimensions. Test samples show slightly higher density with **24.28 non-zero elements** (10.46%). This sparsity has practical implications: memory-efficient sparse matrix representations can be leveraged, and classifiers designed for high-dimensional sparse data become particularly suitable choices.

### 2.4 Classifier Selection and Training

**Classifier and hyperparameters. Logistic regression** is adopted as the baseline classifier for its computational efficiency on sparse high-dimensional inputs and its transparency: the learned coefficients provide a direct indication of which API calls are associated with either class. We use the `liblinear` solver, which is well suited to small and sparse binary classification problems, with the default L2 regularization strength (`C=1.0`), no class weighting, and `random_state=42` for reproducibility. We deliberately avoid extensive hyperparameter tuning at this stage: the goal is to obtain a simple, reproducible reference point rather than optimize the baseline aggressively.

A baseline is not a formality. It anchors the comparison with the more complex architectures in the following tasks and prevents attributing improvements to architectural sophistication when the same gains could be achieved by a simpler representation or by exploiting label imbalance.

The data is split into 60% training and 40% validation using stratified sampling to preserve class proportions. Training produces a log-loss of 0.0806 on the training set and 0.1225 on the validation set. The gap between these values suggests mild overfitting, but remains acceptable for a baseline model using only default L2 regularization and no task-specific tuning.

### 2.5 Evaluation and Results

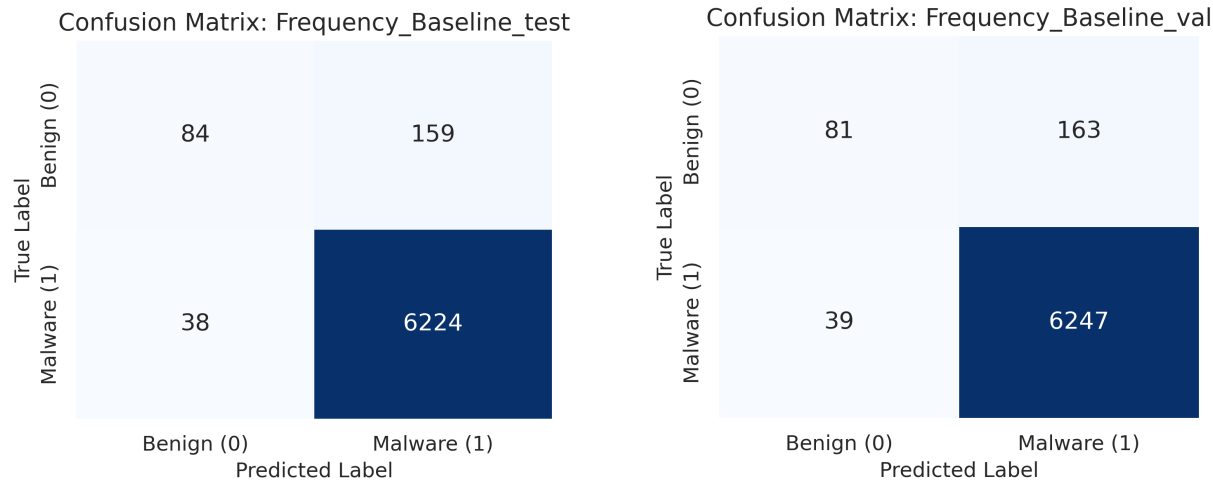
We evaluate the trained classifier on both validation and held-out test sets. Table 1 presents the full classification report for the test set.

Table 1. Frequency-based baseline: classification report (test set)

| Class/Metric | Precision | Recall | F1-Score | Accuracy | Support |
|---|---|---|---|---|---|
| Benign (0) | 0.6885 | 0.3457 | 0.4603 | – | 243 |
| Malware (1) | 0.9751 | 0.9939 | 0.9844 | – | 6,262 |
| **Overall** | – | – | – | 0.9697 | 6,505 |
| **Macro avg** | 0.8318 | 0.6698 | 0.7223 | – | 6,505 |
| **Weighted avg** | 0.9644 | 0.9697 | 0.9648 | – | 6,505 |

**Baseline performance.** The results reveal a markedly asymmetric behavior driven by the underlying imbalance. Malware detection is extremely strong (precision 97.51%, recall 99.39%, F1-score 0.9844), and the confusion matrices in Figure 1 confirm that false negatives are rare. The benign class is more challenging: although benign predictions are often correct (precision 68.85%), recall is low (34.57%), meaning that a large fraction of benign samples are flagged as malware. With only 3.74% benign samples, the model learns a conservative decision boundary that favors catching malware at the cost of additional false alarms.

Despite ignoring order and operating on sparse vectors, the frequency-based baseline reaches **96.97% accuracy** and near-perfect malware recall. This shows that the presence and relative frequency of specific API calls already carries substantial discriminative signal, even before exploiting sequential dependencies. At the same time, the **macro**




(a) Confusion matrix for the frequency-based baseline on the test set.

(b) Confusion matrix for the frequency-based baseline on the validation set.

Fig. 1. Comparison of confusion matrices for the frequency-based baseline on test and validation sets.

**F1-score of 0.7223** and the **benign recall of 34.57%** make clear that the model is not equally reliable across classes: its high accuracy is mainly driven by the dominant malware class.

The limitation is structural: frequency vectors cannot represent temporal motifs, repeated call patterns, or long-range dependencies. This motivates the sequence- and structure-aware models in the subsequent tasks, where the representation is designed to preserve more of the behavioral dynamics that are erased by the bag-of-words transformation.

## 3 TASK 2: FEED FORWARD NEURAL NETWORK (FFNN)

The Feed-Forward Neural Network (FFNN) represents the first deep learning architecture applied to the detection task, moving beyond bag-of-words counts to a positional representation of API calls. This transition introduces two primary constraints: the requirement for fixed-size input vectors and the lack of explicit temporal modeling.

### 3.1 Sequence Length Variability and Fixed-Size Encoding

**Sequence-length variability.** The API call traces exhibit **significant variability in length**. An analysis of the training set shows a distribution ranging from 60 to 90 calls (mean 75.03), while the test set is shifted toward higher values, ranging from 70 to 100 calls (mean 86.33). This mismatch is visualized in Figure 2. Therefore, the **test set contains sequence lengths never observed during training**, which is particularly relevant for a fixed-input architecture.

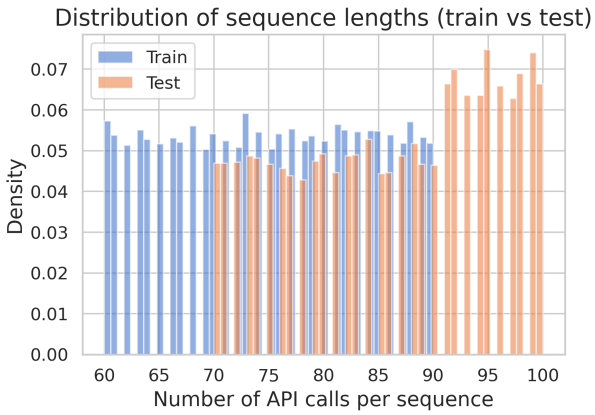



Fig. 2. Distribution of API call sequence lengths in training and test sets.

**Fixed-size encoding.** To accommodate the fixed input dimension of the FFNN, we established a target length of **90** based on the maximum observed in the training data. This choice ensures that no training information is lost via truncation. Sequences shorter than 90 are extended using **zero-padding**, while test sequences exceeding this limit are **truncated**.

The fixed length is estimated only from the **training partition**, in order to avoid using information from the held-out test set to shape the preprocessing pipeline. Alternative choices such as the median length (75) or the 75th percentile (83) would reduce memory usage, but would also discard part of the training sequences. Since the maximum training length is not an extreme outlier, we choose the conservative value of 90.

### 3.2 Categorical Encoding

**Categorical mapping.** Before feeding sequences to the network, API names must be mapped into a numerical representation. The vocabulary is extracted from the padded training sequences, assigning index 0 to `<PAD>` and index 1 to `<UNK>`; the resulting vocabulary contains 260 entries, including the two special tokens. Unseen API calls in the test set are mapped to `<UNK>`.

We evaluate two alternatives. In the **sequential ID** setting, each API call is represented directly by its integer identifier, producing a 90-dimensional input vector. This is simple but problematic: the model can interpret IDs as ordered numerical quantities, even though the numbering is arbitrary. In the **learnable embedding** setting, each API ID is first mapped to a trainable vector, and the 90 embedded calls are flattened before entering the dense layers. This gives the model the opportunity to learn a more meaningful representation of API calls.

### 3.3 Architectural Design and Hyperparameters

**Model alternatives.** We implemented two variants of the FFNN to evaluate the impact of input representation:

(1) **Sequential IDs:** Raw integer identifiers are fed directly into a deep network with hidden dimensions of $[256, 128, 64]$. This model treats IDs as ordinal values, which is an architectural limitation.
(2) **Learnable Embeddings:** Each API ID is mapped to a continuous vector space. This allows the model to learn semantic similarities between calls. The hidden layers were set to $[128, 64, 32]$.

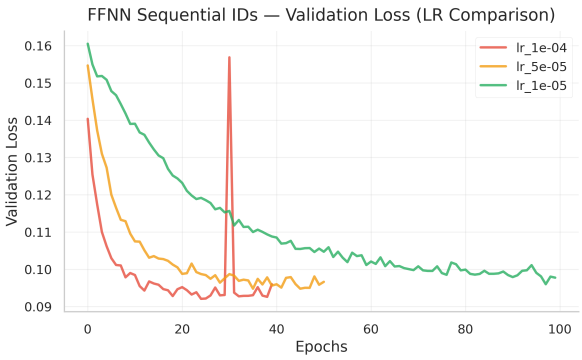
Both models utilize **Batch Normalization** for training stability and **Dropout** (0.3–0.4) to mitigate overfitting. Training is performed with **AdamW**, **BCEWithLogitsLoss**, batch size 64, and early stopping with patience 15 on validation loss. Since label 1 corresponds to malware and is the majority class, the loss uses `pos_weight=0.20` to down-weight malware errors and prevent the minority benign class from being completely overwhelmed.

**Hyperparameter selection.** For the neural models, the original training partition is split into 80% training and 20% validation using stratified sampling, preserving the class proportions. For the sequential-ID model, we test learning rates $\{10^{-4}, 5 \cdot 10^{-5}, 10^{-5}\}$ with weight decay $10^{-5}$. For the embedding model, we test $\{5 \cdot 10^{-5}, 10^{-5}, 5 \cdot 10^{-6}\}$ with weight decay $10^{-4}$. After selecting the learning rate, we also test embedding sizes $\{8, 16, 32, 64, 128\}$ to isolate the effect of representation capacity. The FFNN configurations are selected by the **minimum validation loss**, using macro F1 only as a qualitative class-balance check; for the later RNN and GNN models, where training is noisier and balanced behavior becomes the main objective, validation macro F1 is used as the primary selection criterion.

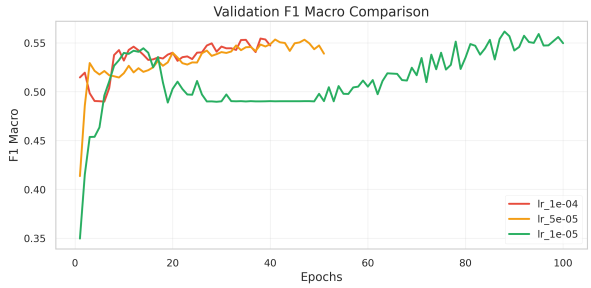
The plots in Figure 3 are used to check the overall convergence behavior rather than to read exact values. For the sequential-ID model, the validation-loss curves show a faster convergence for the larger learning rates, and $lr = 10^{-4}$ is selected because it gives the lowest validation loss. The macro F1 curves are close and noisy, so the exact comparison is left to the logged metrics. For the embedding model, $lr = 5 \cdot 10^{-5}$ reaches the best selected validation loss and a visibly stronger macro-F1 region than the slower learning rates.

### 3.4 Results and Discussion

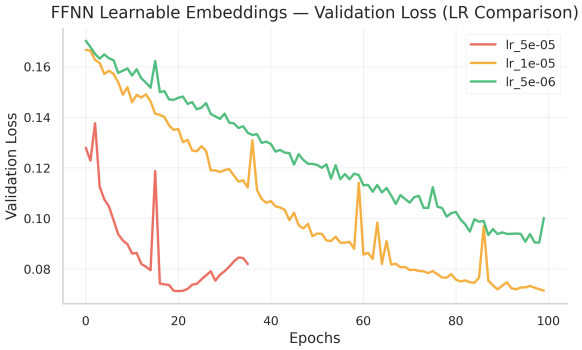
**Sequential IDs versus embeddings.** Table 2 summarizes the selected models. The **embedding-based FFNN** obtains a lower validation loss and better validation macro F1 than the sequential-ID model. The plotted curves are consistent with this conclusion at a high level, but the exact comparison comes from the logged histories reported in the table. The sequential-ID representation remains fragile because the network can exploit accidental numerical magnitudes of API identifiers; embeddings reduce this artifact by learning continuous API-call representations. On the test set, embeddings improve macro F1 from **0.5452 to 0.5789**, although the benign class remains difficult because the model still receives a fixed, truncated vector rather than a temporal trace.



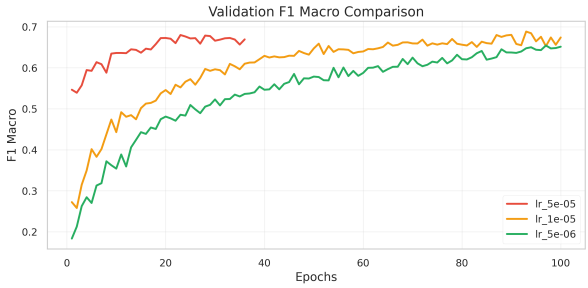

(a) Sequential IDs: validation loss.




(b) Sequential IDs: macro F1.




(c) Embeddings: validation loss.




(d) Embeddings: macro F1.

Fig. 3. Validation loss and macro F1 behavior of the FFNN variants across learning rates.

Table 2. Task 2 FFNN results for the selected configurations.

| Model | LR | Min Val Loss | Val Macro F1 | Test Acc. | Test Macro F1 |
|---|---|---|---|---|---|
| Sequential IDs | $10^{-4}$ | 0.0921 | 0.5400 | 0.9333 | 0.5452 |
| Learnable Embeddings | $5 \cdot 10^{-5}$ | 0.0712 | 0.6709 | 0.9279 | 0.5789 |

The embedding-size sweep in Table 3 shows the expected trend: very small embeddings underfit the API representation, while larger embeddings improve validation behavior. The lowest validation loss is obtained with **128 dimensions**, while the best validation and test macro F1 are obtained with **64 dimensions**. Since the differences between the larger embedding sizes are modest, the result should be interpreted as a capacity trend rather than as evidence that one exact dimensionality is universally optimal.

Table 3. Embedding-size sensitivity for the learnable-embedding FFNN.

| Emb. dim | Min Val Loss | Val Macro F1 | Test Macro F1 | Test Benign Rec. |
|---|---|---|---|---|
| 8 | 0.0885 | 0.5847 | 0.5382 | 0.2181 |
| 16 | 0.0823 | 0.6038 | 0.5568 | 0.2510 |
| 32 | 0.0787 | 0.6368 | 0.5705 | 0.2593 |
| 64 | 0.0741 | 0.6756 | 0.5902 | 0.2510 |
| 128 | 0.0730 | 0.6440 | 0.5740 | 0.2881 |

Overall, both FFNN variants remain **below the frequency-based baseline** of Task 1 in macro F1. The main reason is architectural: after padding or truncation, the FFNN cannot model the temporal evolution of the API trace, and **40% of test sequences are longer than 90 calls**, so useful late-sequence behavior may be removed. This motivates the recurrent models in the next task.

# 4 TASK 3: RECURRENT NEURAL NETWORK (RNN)

This section investigates recurrent neural networks for malware classification from ordered API call traces. Differently from the FFNNs in Task 2, recurrent models process the sequence step by step and can exploit temporal dependencies without forcing all samples into the same global length.

## 4.1 Sequence Preprocessing

**Dynamic padding.** RNNs still require padding to build mini-batches, but padding is handled **dynamically rather than globally**. We use a custom `collate_fn` that sorts the samples in each batch by decreasing length, pads them only up to the longest sequence in that batch, and returns both the padded tensor and the original lengths. The embedding layer reserves index 0 for padding through `padding_idx=0` , and `pack_padded_sequence` is used before the recurrent layer. In this way, the model does not update its hidden state on dummy positions and avoids wasting computation on padding tokens.

**Variable-length inference.** This preprocessing also **removes the need for test-time truncation**. A recurrent network shares the same parameters across time steps, therefore longer traces simply imply additional recurrent updates. This is an important difference with respect to Task 2, where test sequences longer than 90 API calls had to be truncated to match the FFNN input size. As a consequence, recurrent models can preserve late API calls that may contain useful behavioral information.

From a memory perspective, dynamic padding makes the batch tensor proportional to the longest sequence in the current batch, not to a fixed global maximum. Moreover, recurrent models keep a compact 32-dimensional embedding for each API call and process it sequentially, instead of flattening the whole embedded sequence into a large dense vector as in the embedding-based FFNN. The price is computational speed: the recurrent dependency between time steps reduces parallelization and makes training **slower than FFNN training**.

## 4.2 RNN Architectures and Model Selection

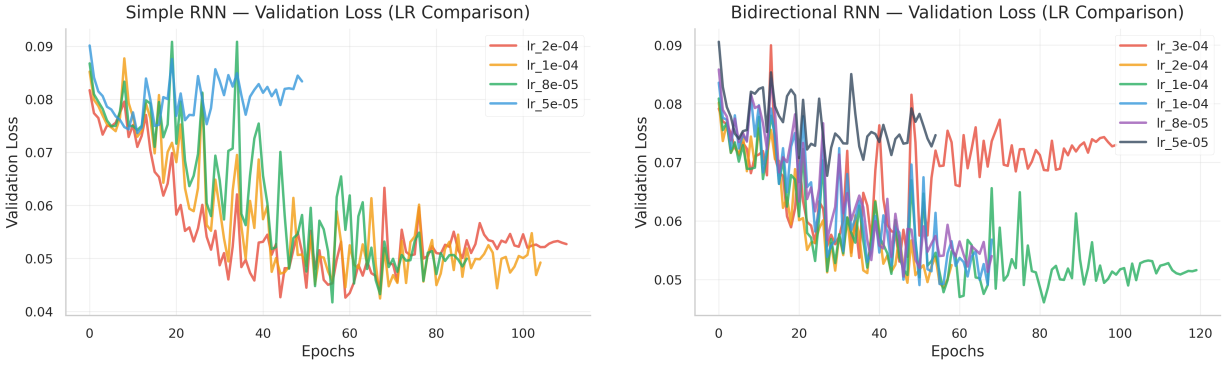
**Architecture search.** We trained three recurrent variants: a simple one-directional RNN, a bidirectional RNN, and a bidirectional LSTM. All models use a vocabulary built from the training set, 32-dimensional embeddings, batch size 64, AdamW optimization, binary cross-entropy with logits, early stopping, and a plateau-based learning-rate scheduler. Since label 1 corresponds to malware and is the majority class, `pos_weight=0.20` down-weights malware errors and helps the minority benign class influence the optimization. The model-specific choices are summarized in Table 4.

Table 4. RNN architecture and learning-rate configurations.

| Model | Dir. | Hidden | Dropout | Learning rates tested | Sel. LR | Val. loss | Val. Macro F1 |
|---|---|---|---|---|---|---|---|
| Simple RNN | one-way | 128 | 0.30 | 0.00020, 0.00010, 0.00008, 0.00005 | 0.00020 | 0.0567 | 0.8601 |
| Bi-RNN | two-way | 128 | 0.30 | 0.00005, 0.00008, 0.00010, 0.00015, 0.00020, 0.00030 | 0.00010 | 0.0498 | 0.8222 |
| Bi-LSTM | two-way | 128 | 0.30 | 0.00005, 0.00008, 0.00010, 0.00015, 0.00020, 0.00030 | 0.00030 | 0.0437 | 0.8722 |

The Simple RNN uses two vanilla recurrent layers with `tanh` activation. The bidirectional RNN keeps the same cell type but processes each sequence in both directions, while the bidirectional LSTM replaces vanilla recurrent units with gated memory cells to improve stability on longer dependencies.

The final configurations are selected primarily by validation macro F1, with validation loss used to verify that the selected checkpoint is not the result of an unstable spike.




(a) Simple RNN. (b) Bi-RNN.

Fig. 4. Validation loss curves for the vanilla recurrent architectures across learning rates.

The plots are used only as a qualitative check of convergence. In Figure 4, the vanilla recurrent models show visibly noisy validation-loss trajectories, and the lower learning-rate runs tend to keep decreasing for longer. For the LSTM in Figure 5, $lr = 0.0003$ reaches the lowest visible loss band among the tested settings. The exact selection values are reported in Table 4: the **bidirectional LSTM has both the highest validation macro F1 and the lowest validation loss** among the selected recurrent configurations.

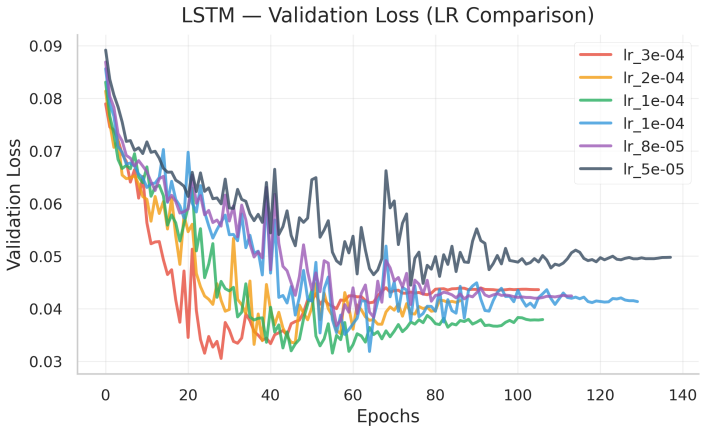



Fig. 5. Validation loss curve for the bidirectional LSTM across learning rates.

RNNs are **significantly slower than the FFNNs** from Task 2. The selected FFNN configurations train in roughly half a minute, while the selected Simple RNN requires about 5 minutes and 19 seconds, the bidirectional RNN about 8 minutes and 9 seconds, and the bidirectional LSTM about 8 minutes and 5 seconds. This overhead is caused by recurrent step-by-step processing, backpropagation through time, dynamic sorting and padding inside the data loader, and packed-sequence handling.

## 4.3 Performance Analysis

Table 5. Performance comparison on the test set.

| Model | LR | Accuracy | Benign (0) | | Malware (1) | | Macro F1 |
|---|---|---|---|---|---|---|---|
| | | | Prec. | Rec. | Prec. | Rec. | |
| Simple RNN | 2e-4 | 97.48% | 0.6626 | 0.6626 | 0.9869 | 0.9869 | 0.8247 |
| Bi-RNN | 1e-4 | 96.91% | 0.5734 | 0.6749 | 0.9873 | 0.9805 | 0.8020 |
| Bi-LSTM | 3e-4 | 97.57% | 0.6763 | 0.6708 | 0.9872 | 0.9875 | 0.8305 |
| Freq. baseline | – | 96.97% | 0.6885 | 0.3457 | 0.9751 | 0.9939 | 0.7223 |
| Seq. ID FFNN | 1e-4 | 93.33% | 0.1225 | 0.1276 | 0.9661 | 0.9645 | 0.5452 |
| Embed. FFNN | 5e-5 | 92.79% | 0.1676 | 0.2346 | 0.9698 | 0.9548 | 0.5789 |

**RNN performance.** Table 5 compares the selected recurrent models with the previous baselines. Even the Simple RNN raises benign recall from 23.46% for the embedding FFNN to **66.26%**, confirming that the **order of API calls contains useful information** lost by fixed-window FFNNs.

In this run, the bidirectional RNN does not improve over the Simple RNN: it keeps a similar benign recall, but loses benign precision and therefore obtains a lower macro F1. The bidirectional LSTM is the best recurrent trade-off: its gates stabilize the recurrent state and help preserve relevant information over longer API traces, producing the highest validation and test macro F1 among the Task 3 models.

The validation curves in Figures 4 and 5 show that recurrent training is **less smooth than FFNN training**. Small changes in learning rate produce visible changes in convergence, and vanilla RNNs are more exposed to vanishing or unstable gradients. The figures are used to assess the shape of convergence, while exact F1 values come from the logged histories and from Table 5. The important visual trend is that the LSTM reaches a lower validation-loss region than the vanilla recurrent models.

The frequency baseline is very strong on the majority malware class, but its benign recall is only 34.57%, meaning that many benign samples are classified as malware. The bidirectional LSTM improves accuracy from 96.97% to 97.57%, and **improves macro F1 from 0.7223 to 0.8305** and benign recall from 34.57% to 67.08%. In an imbalanced setting,

this is a better balanced behavior: the model keeps majority-class performance high while substantially reducing the asymmetry between the two classes.

## 5 TASK 4: GRAPH NEURAL NETWORK (GNN)

The last modeling strategy represents each API call sequence as a graph. While RNNs preserve the execution trace as a temporal chain, GNNs reinterpret the same behavior as a set of API calls connected by transitions. This allows the model to reason on local neighborhoods and repeated structural patterns, rather than only on a fixed vector or on a left-to-right hidden state.

### 5.1 Graph Construction and Preprocessing

**Graph construction.** Each sample is converted into a transition graph. The nodes correspond to the unique API calls appearing in the sequence, while **unweighted edges** connect consecutive calls with window size 1. Therefore, a transition such as $A \rightarrow B$ creates an edge from the local node representing $A$ to the local node representing $B$; loops are naturally allowed when the same API call follows itself. Node features are the global API identifiers, which are then mapped to 32-dimensional learnable embeddings inside the GNN. Unknown API calls are mapped to the `<UNK>` index, and padding is excluded from the graph construction.

**Variable-size batching.** This representation **removes the need for padding**. Unlike FFNNs and RNNs, PyTorch Geometric batches graphs by forming a larger disconnected graph and keeping a `batch` vector that records the graph membership of each node. As a result, each sample can keep its own number of nodes and edges, and the model produces a graph-level representation through `global_mean_pool`. In the first training graph, for example, only 21 unique API-call nodes and 80 transition edges are needed, even though the global vocabulary contains 260 entries.

The same argument explains why **test sequences are not truncated**. Longer traces simply generate additional transitions and possibly additional nodes. Truncating them would remove edges from the behavioral graph and would partially defeat the reason for using a graph representation. This is a clear advantage over the FFNN setting of Task 2, where sequences longer than 90 calls had to be cut.

Compared with RNNs, GNNs can **aggregate information from neighborhoods** and capture recurring transition motifs, such as an API call connected to several different successors or repeated cycles between calls. This can make the representation more robust to small local reorderings. However, this comes at a cost: after graph construction and global pooling, the model no longer knows exactly whether a transition occurred at the beginning or at the end of the trace. Thus, GNNs preserve structural relations between API calls, but **lose part of the strict chronological information** explicitly modeled by RNN hidden states.

### 5.2 GNN Architectures and Model Selection

**Graph architectures.** We trained three graph architectures: GCN, GraphSAGE, and GAT. All models share the same basic setup: 32-dimensional embeddings, hidden dimension 64, three graph convolution/message-passing layers, global mean pooling, batch size 32, AdamW optimization, early stopping, and binary cross-entropy with logits. As in the previous neural models, the loss uses `pos_weight=0.20`, since malware is the positive and majority class; this down-weights malware errors and gives more relative influence to the minority benign samples.

The GCN model uses three `GCNConv` layers followed by ReLU activations and dropout $p = 0.2$, then a small classifier with dropout $p = 0.5$. GraphSAGE replaces the convolutional operator with three `SAGEConv` layers, allowing node representations to be updated through neighborhood aggregation. GAT uses three attention-based layers: the first `GATConv` uses 4 attention heads, while the following layers use a single head before the graph-level pooling. For GCN and GraphSAGE we tested learning rates $\{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}\}$, while for GAT we tested $\{5 \cdot 10^{-5}, 10^{-4}, 5 \cdot 10^{-4}\}$.

The final configurations are selected using validation behavior, consistently with the previous tasks.

Table 6. GNN learning-rate search and selected configurations.

| Model | Learning rates tested | Sel. LR | Val. Macro F1 | Test Macro F1 |
|---|---|---|---|---|
| GCN | 0.0001, 0.0005, 0.001 | 0.001 | 0.8010 | 0.7654 |
| GraphSAGE | 0.0001, 0.0005, 0.001 | 0.001 | 0.8236 | 0.8079 |
| GAT | 0.00005, 0.0001, 0.0005 | 0.0005 | 0.7932 | 0.7962 |

All models use 32-dimensional node embeddings, hidden dimension 64, three message-passing layers, dropout $p = 0.2$ in the graph encoder and $p = 0.5$ in the classifier, batch size 32, AdamW optimization, early stopping, global

mean pooling, and binary cross-entropy with logits using `pos_weight=0.20`. GAT uses four attention heads in the first layer and one head in the following layers.

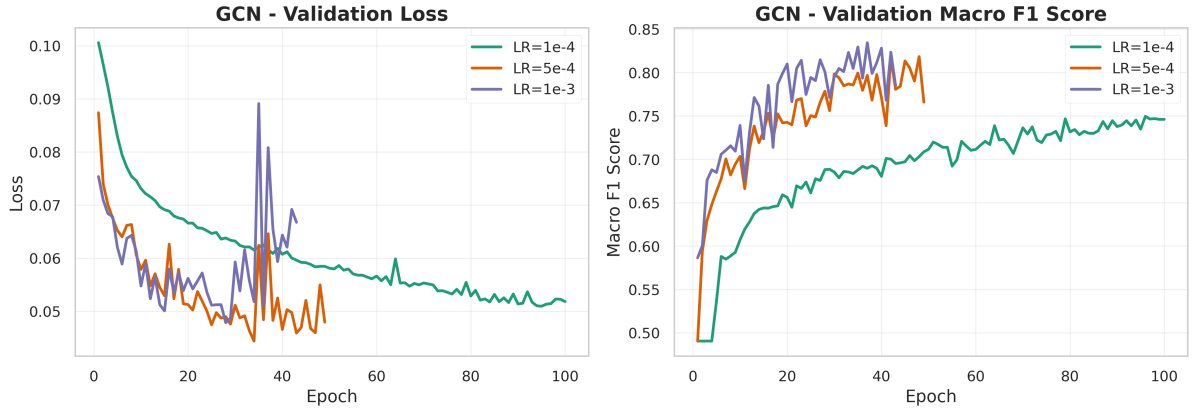



Fig. 6. Validation curves for GCN across learning rates.

The curves are interpreted as qualitative convergence checks, while the exact selection values are reported in Table 6. For GCN, $lr = 0.001$ reaches the upper validation-F1 region, but its loss curve is visibly noisier than the lower-rate runs.

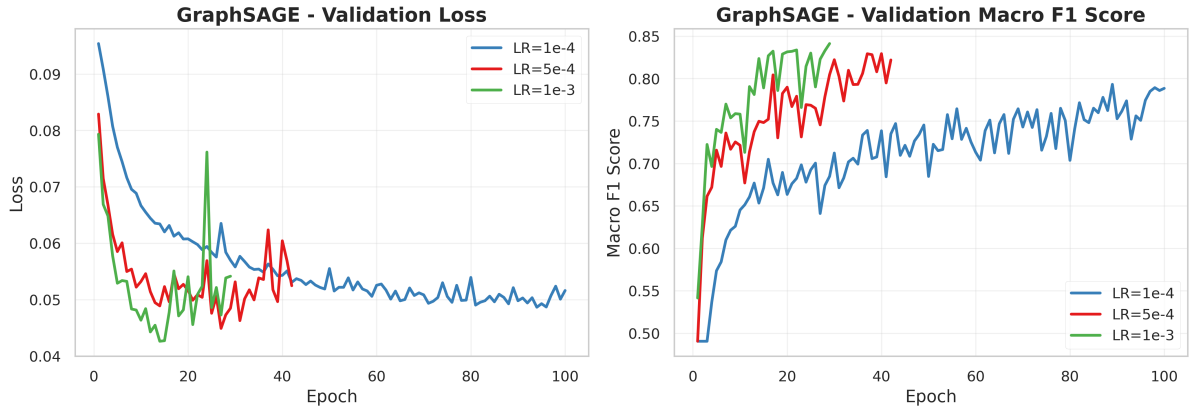



Fig. 7. Validation curves for GraphSAGE across learning rates.

For GraphSAGE, the $lr = 0.001$ curve moves quickly into the highest visible macro-F1 band and keeps the loss broadly competitive. The logged metrics in Table 6 confirm this as the strongest GNN configuration, with the highest validation macro F1 and the best GNN test macro F1.

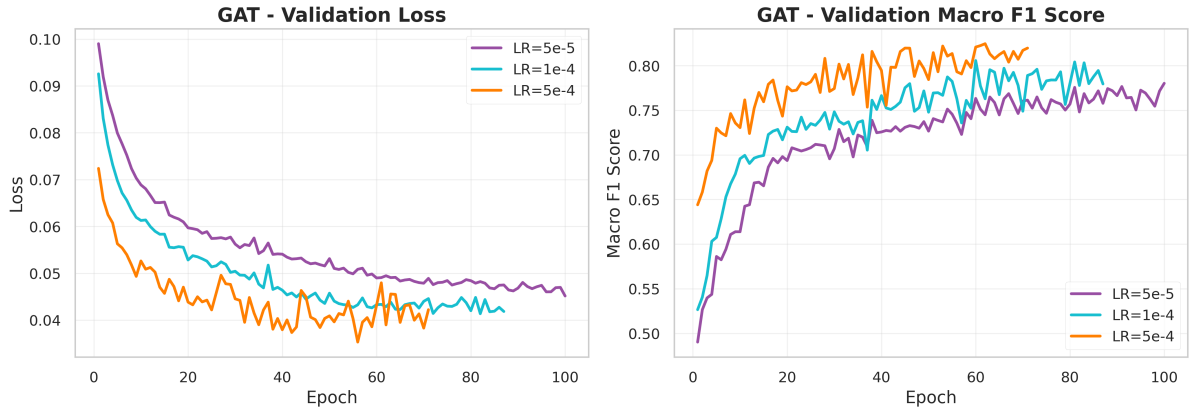



Fig. 8. Validation curves for GAT across learning rates.

For GAT, $lr = 0.0005$ shows the fastest early gain, while the lower learning rates look smoother but slower. The exact validation and test values are therefore taken from the logged histories in Table 6, not estimated from the plotted curves.

## 5.3 Performance Analysis

Table 7 provides the final comparison across all selected architectures. The GNNs clearly **outperform both FFNN variants** from Task 2, confirming that transition structure is much more informative than fixed positional vectors. They also **improve over the frequency baseline in macro F1**, although the baseline remains very competitive in accuracy because it is extremely strong on the majority malware class.

Table 7. Final performance comparison on the test set.

| Model | LR | Accuracy | Benign (0) | | Malware (1) | | Macro F1 |
|---|---|---|---|---|---|---|---|
| | | | Prec. | Rec. | Prec. | Rec. | |
| GCN | 1e-3 | 96.05% | 0.4788 | 0.6502 | 0.9862 | 0.9725 | 0.7654 |
| GraphSAGE | 1e-3 | 96.99% | 0.5813 | 0.6914 | 0.9879 | 0.9807 | 0.8079 |
| GAT | 5e-4 | 96.40% | 0.5125 | 0.7572 | 0.9904 | 0.9721 | 0.7962 |
| Simple RNN | 2e-4 | 97.48% | 0.6626 | 0.6626 | 0.9869 | 0.9869 | 0.8247 |
| Bi-RNN | 1e-4 | 96.91% | 0.5734 | 0.6749 | 0.9873 | 0.9805 | 0.8020 |
| Bi-LSTM | 3e-4 | 97.57% | 0.6763 | 0.6708 | 0.9872 | 0.9875 | 0.8305 |
| Freq. baseline | – | 96.97% | 0.6885 | 0.3457 | 0.9751 | 0.9939 | 0.7223 |
| Seq. ID FFNN | 1e-4 | 93.33% | 0.1225 | 0.1276 | 0.9661 | 0.9645 | 0.5452 |
| Embed. FFNN | 5e-5 | 92.79% | 0.1676 | 0.2346 | 0.9698 | 0.9548 | 0.5789 |

**Graph-level comparison.** Among the selected GNN models, **GraphSAGE is the strongest graph architecture**: its macro F1 0.8079 is the highest among GNNs because it balances benign precision and recall better than GCN and GAT. GAT reaches the highest benign recall among graph models, but with lower benign precision; all graph models keep excellent malware precision and recall, around 98.6%–99.0% precision and above 97% recall.

Compared with the baseline, GraphSAGE keeps essentially the same accuracy while increasing macro F1, showing that transition structure improves balanced classification without losing majority-class behavior. The bidirectional LSTM remains the best model overall in this run, with macro F1 0.8305, but GraphSAGE is close and clearly stronger than the frequency and FFNN baselines. The preferred architecture still depends on the desired precision–recall trade-off: GAT recalls more benign samples, while GraphSAGE gives the best graph-level balance.

**Architecture trade-off.** The final ranking also shows why accuracy is not sufficient for this dataset. The frequency baseline reaches 96.97% accuracy, almost the same level as the best neural models, but its benign recall is only 34.57%. In a malware-analysis setting this means that the model is very effective at recognizing the dominant malware class, but it tends to over-flag benign software. Sequence-aware and graph-aware models reduce this asymmetry: Bi-LSTM reaches 67.08% benign recall, while the GNNs remain between 65.02% and 75.72%. Therefore, the main gain of the complex architectures is not a large increase in accuracy, but a more balanced decision boundary.

**RNNs versus GNNs.** The Bi-LSTM is the strongest overall model in terms of test macro F1, suggesting that the exact temporal order of API calls is highly informative. However, GraphSAGE obtains a close macro F1 with a different inductive bias: it focuses on local transition neighborhoods and repeated API-call structures rather than on the full left-to-right execution trace. This makes the graph representation attractive when the same behavioral pattern may appear at different positions in the trace. The cost is that global chronology is compressed by graph pooling, so the model may miss patterns where the precise timing of a call is decisive.

Overall, the experiments support a clear progression. Frequency vectors are a strong and interpretable reference, FFNNs struggle because fixed-size truncation and raw positional encoding discard important behavior, RNNs recover temporal dependencies, and GNNs provide a competitive structural alternative without padding or truncation. For this run, the preferred final model is the **Bi-LSTM** when optimizing the global macro F1, while **GraphSAGE** is the best graph-based solution and the strongest alternative when transition structure is the desired representation.